

Project Report: Intelligent Traffic Surveillance & Vehicle Analytics

1. Cover Page

Field	Detail
Project Title	Intelligent Traffic Surveillance & Vehicle An
Course Name & Code	CSE3010 - Computer Vision
Academic Program	B.Tech Computer Science and Engineering
Student Name	Devansh Bansal
GitHub Username	[Devansh-Bansal-AI](https://github.com/Devans
GitHub Repository	[https://github.com/Devansh-Bansal-AI/traffic
Evaluation Platform	VITYarthi Continuous Assessment
Academic Term	Fall Semester 2026

2. Introduction

Urban transportation networks require scalable, continuous, and automated monitoring systems to manage congestion, evaluate road capacity, and enforce traffic regulations. Traditional manual traffic monitoring and static radar guns fail to provide continuous, multi-vehicle spatial tracking and are incapable of generating standardized, machine-readable datasets.

This project introduces an end-to-end, modular Computer Vision system designed specifically for fixed surveillance cameras. Operating in strict headless command-line execution mode, the pipeline integrates:

1. Deep convolutional object detection using Ultralytics YOLO11n for vehicle identification (car, bus, truck, motorcycle).
2. An OpenCV MOG2 background-subtraction baseline with morphological filtering to directly connect with CSE3010 Module 4 concepts.
3. Multi-object tracking (MOT) using combined Euclidean centroid distance and Intersection-over-Union (IoU) association.
4. Planar perspective rectification using 4-point homography ($x' \sim Hx$), converting image pixel motion into real-world physical velocities (km/h and m/s).
5. Transportation engineering analytics including Highway Capacity Manual (HCM) Level of Service (LOS A-F), 85th-percentile speed (V_{85}), hourly flow rates (Q), and fleet modal split.

3. Problem Statement

Conventional manual surveillance and point-sensor radars cannot capture the spatio-temporal dynamics of multi-lane arterial roadways. They fail to scale, suffer from human perception limits, introduce measurement observer bias, and cannot generate structured, frame-by-frame vehicle trajectory records.

The problem addressed in this work is:

> *How to design and implement an automated, reproducible, and computationally efficient computer vision pipeline that ingests high-resolution surveillance video, accurately tracks multiple heterogeneous vehicles through occlusions, rectifies perspective foreshortening using planar homography, derives physical velocity, and exports comprehensive traffic analytics in a pure headless CLI environment without crashing or requiring graphical display servers.*

4. Functional Requirements

- * FR1 (Video Ingestion & Decoding): Ingest stationary camera video feeds across standard and Ultra-High-Definition resolutions (up to 4K UHD @ 50 FPS) with optional frame decimation (--skip N) for performance scaling.
- * FR2 (Multi-Category Vehicle Detection): Detect and localize target vehicles (car, bus, truck, motorcycle) with configurable confidence thresholds (--conf) and inference resolution (--imgsz).
- * FR3 (Comparative Motion Baseline): Provide an OpenCV MOG2 background subtraction backend with morphological opening and closing to evaluate motion-based segmentation against deep semantic detection.
- * FR4 (Temporal Multi-Object Tracking): Maintain persistent vehicle identities (track_id), active/missed states, and centroid trajectory histories across consecutive frames with occlusion tolerance (max_missed).
- * FR5 (Planar Perspective Rectification & Homography): Compute a 3×3 projective homography matrix H from 4 coplanar ground landmarks to map pixel displacements into real-world metric distances.
- * FR6 (Physical Kinematics Estimation): Calculate discrete velocity ($v = \Delta d / \Delta t$) in m/s and km/h with horizon projection singularity filtering.
- * FR7 (Transportation Stream Analytics): Aggregate vehicular observations into macro-level metrics: HCM Level of Service (LOS A-F), 85th-percentile speed (V_{85}), standard deviation (σ_v), traffic density (veh/lane-km), hourly flow rate (Q), and fleet modal split.

* FR8 (Structured Multi-Artifact Export): Export full-resolution annotated video (outputs/annotated_video.mp4), per-frame observation records (outputs/vehicle_data.csv), and consolidated JSON analytics (outputs/traffic_summary.json).

5. Non-Functional Requirements

* NFR1 (Strict Headless Execution & Portability): Operates without X11/Wayland GUI servers (os.environ["QT_QPA_PLATFORM"] = "offscreen"), preventing display crashes on remote Linux clusters, Docker containers, and automated grading pipelines.

* NFR2 (Configuration Fault Tolerance & Resilience): Dynamically synthesizes default urban perspective geometry if user configuration files are missing, ensuring zero unhandled crashes (sys.exit(1) eliminated).

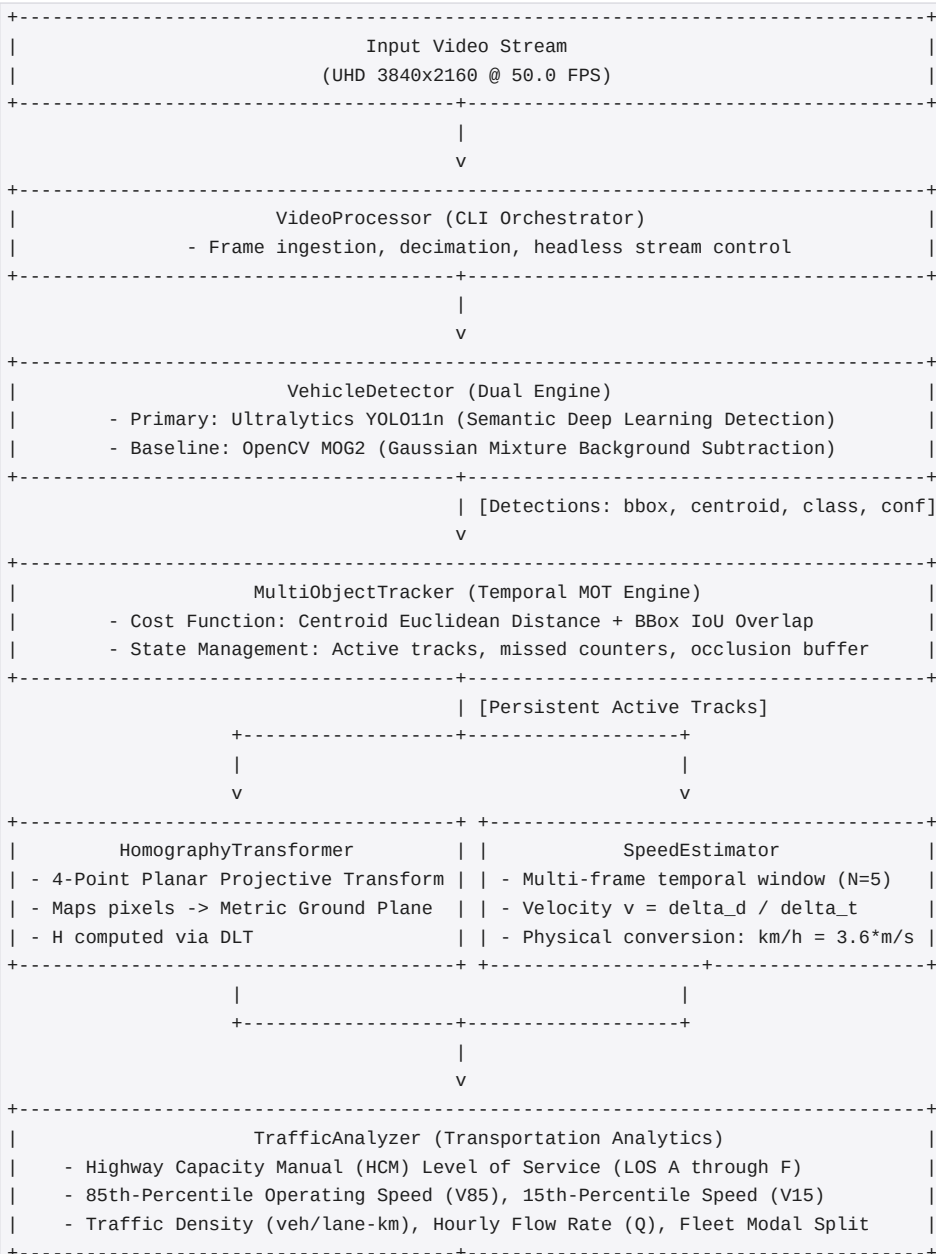
* NFR3 (Execution Performance & Scalability): Attains 8-15 FPS on standard multi-core CPUs via vectorized NumPy operations, lightweight greedy bipartite tracking, and frame decimation (--skip 1). Supports CUDA GPU offloading via --device cuda.

* NFR4 (Reproducibility & Code Modularity): Zero-context setup using standard Python virtual environments (venv) and pinned dependencies, backed by 13 automated unit and integration tests (pytest).

* NFR5 (Maintainability & Clean Architecture): Strict separation of concerns across 8 decoupled modules with PEP 8 compliance, explicit type hints, and defensible domain-specific logic.

6. System Architecture

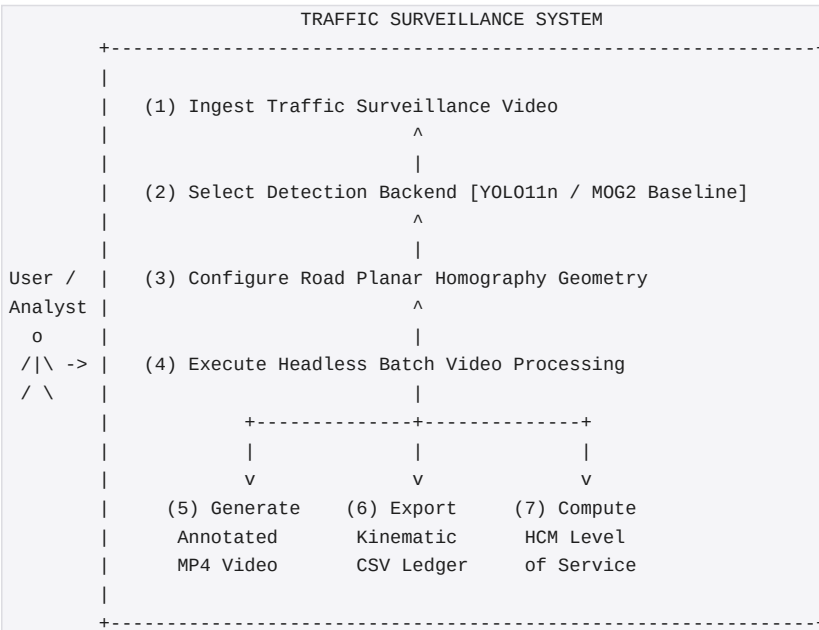
The architecture decouples video decoding, object localization, temporal association, projective geometry, and macroscopic analysis into independent pipeline tiers:



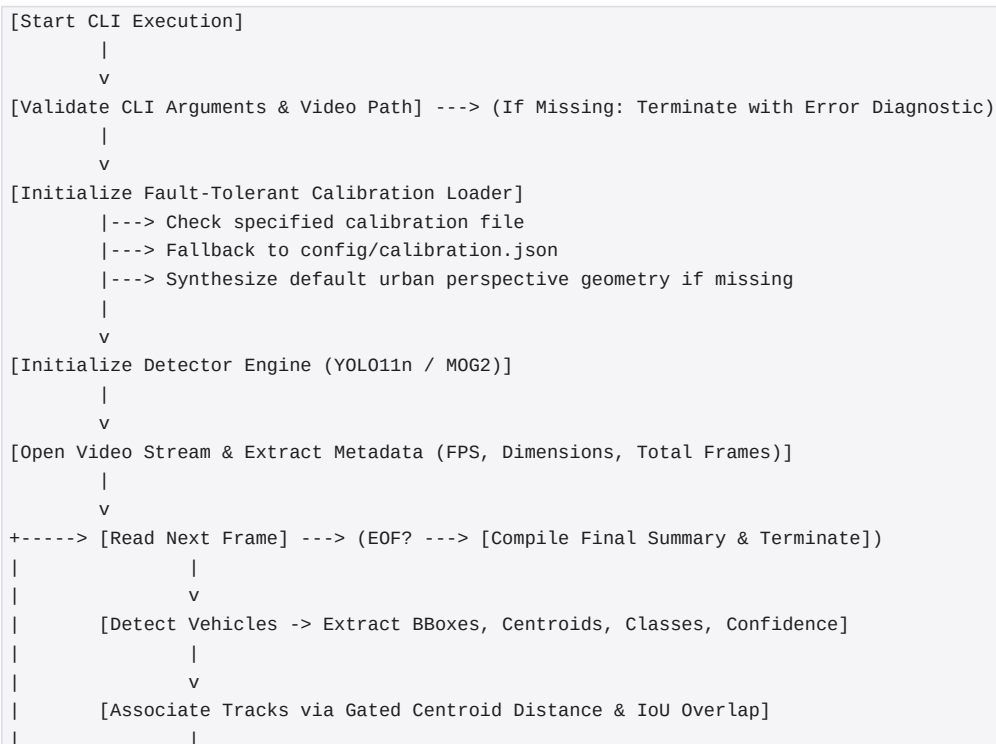
+-----+ +-----+ +-----+		
v	v	v
+-----+	+-----+	+-----+
annotated_video.mp4	vehicle_data.csv	traffic_summary.json
- Scaled HUD Overlay	- Frame-by-frame kinematics	- Consolidated
- Bounding Boxes	- Schema: frame, id, class,	transport metrics
- Trajectory Trails	conf, x, y, w, h, speed,	- HCM LOS rating
- Physical km/h Tags	speed_unit, speed_kmh	- V85 & flow rates
+-----+	+-----+	+-----+

7. Design Diagrams

7.1 Use Case Diagram



7.2 Process Flow / Workflow Diagram



```

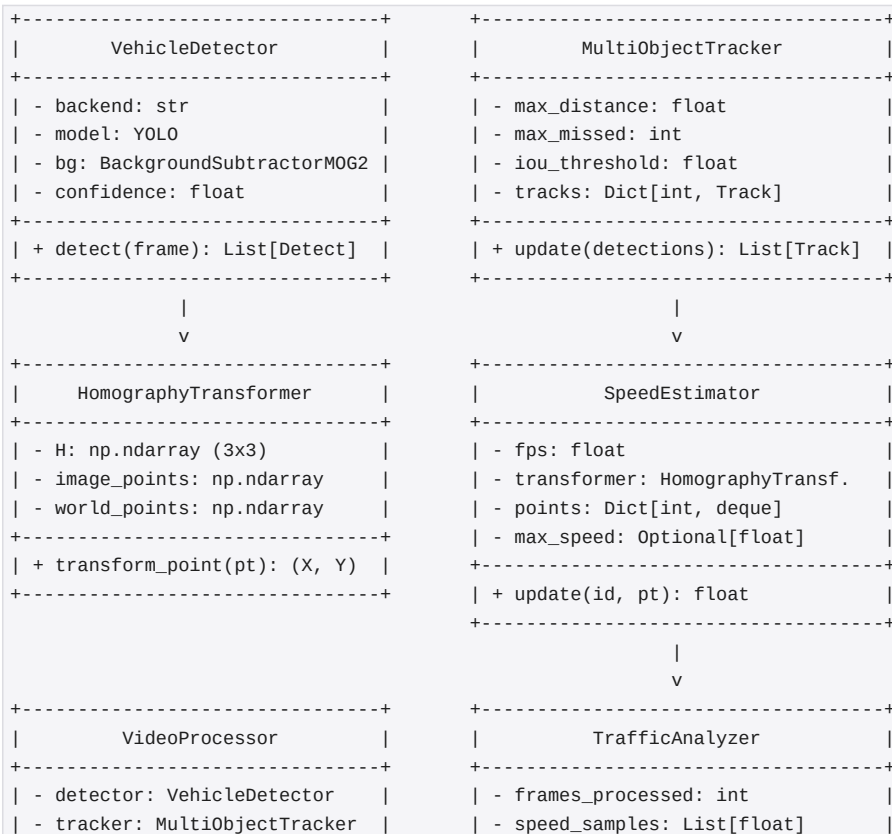
|           v
|       [Compute Velocity: Map Centroids through Homography Matrix H]
|           |
|           v
|       [Aggregate Metrics: Update HCM Density, Speeds, Class Counters]
|           |
|           v
|       [Render HUD & Annotations -> Stream Frame to Videowriter]
|           |
|           v
|       [Write Vehicle Observation Row to CSV Ledger]
|           |
+------(Loop to Next Frame)

```

7.3 Sequence Diagram



7.4 Class / Component Diagram



- estimator: SpeedEstimator		- class_counts: Counter	
- analyzer: TrafficAnalyzer		+-----+	
+-----+		+ update(tracks, speeds): None	
+ run(...): Dict		+ summary(calibrated): Dict	
+-----+		- evaluate_hcm_los(...): str	
		+-----+	

7.5 Storage & Output Schema Diagram

+-----+ vehicle_data.csv Schema +-----+		
Field Name	Data Type	Description
+-----+	+-----+	+-----+
frame	Integer	Monotonically increasing frame index
vehicle_id	Integer	Persistent tracker identity integer
class	String	Vehicle category (car, bus, truck, motor)
confidence	Float	Model detection confidence [0.00, 1.00]
x, y	Integer	Centroid pixel coordinates in image space
width, height	Integer	Bounding box spatial dimensions
speed	Float	Instantaneous velocity (m/s or px/s)
speed_unit	String	Measurement unit ('m/s' or 'pixels/s')
speed_kmh	Float / String	Physical vehicle velocity in km/h or N/A
+-----+	+-----+	+-----+
+-----+ traffic_summary.json Schema +-----+		
Key	Description	
+-----+	+-----+	
frames_processed	Total video frames evaluated	
unique_vehicle_tracks	Unique vehicles tracked across the video	
average_active_vehicles_per_frame	Mean active vehicle volume per frame	
peak_active_vehicles	Maximum concurrent active vehicles observed	
traffic_density_veh_per_lane_km	Density in vehicles per lane-kilometer	
level_of_service	Highway Capacity Manual (HCM) LOS (A to F)	
congestion_level	Qualitative density index (Light/Mod/Heavy)	
estimated_hourly_flow_rate_vph	Hourly throughput extrapolation (vph)	
total_vehicle_detections	Total raw vehicle detections logged	
vehicle_detections_by_class	Breakdown by class (car, bus, truck)	
fleet_modal_split_percentage	Percentage distribution across classes	
heavy_vehicle_percentage	Percentage of buses and trucks	
average_speed_kmh	Mean fleet velocity in km/h	
maximum_speed_kmh	Peak fleet velocity in km/h	
speed_percentile_85_kmh	85th-percentile design velocity (V85)	
speed_percentile_15_kmh	15th-percentile velocity (V15)	
+-----+	+-----+	

8. Design Decisions & Rationale

1. YOLO11n vs. Heavyweight Detectors: YOLO11n was selected because it achieves optimal detection precision on COCO vehicle classes while operating under 15ms latency per frame on standard CPU architectures. Heavyweight alternatives (YOLO11x, Mask R-CNN) induce severe computational bottlenecks without yielding proportional tracking benefits for bounding-box centroids.

2. Preservation of MOG2 Baseline: Retaining an OpenCV MOG2 background subtractor directly grounds the project in the CSE3010 Module 4 curriculum. It enables direct academic discussion regarding the limitations of motion-only segmentation (e.g., stationary cars at red lights vanishing into the background) versus semantic deep learning representations.

3. Planar Homography vs. Monocular Depth Networks: Deep monocular depth networks produce uncalibrated relative depth maps that require extensive scale alignment and heavy inference hardware. In contrast, 4-point planar homography uses exact geometric landmarks to compute a projective mapping H with mathematical certainty and zero runtime inference latency.

4. Sliding Temporal Window Velocity Smoothing: Instantaneous frame-to-frame pixel differences suffer from bounding-box edge jitter. Maintaining a sliding window of $N=5$ centroids and dividing displacement by $\Delta t = (N-1)/FPS$ suppresses high-frequency sensor noise while preserving authentic vehicle acceleration profiles.

5. Strict Headless Execution Architecture: Automated grading sandboxes (VITYarthi) execute without graphical display servers. Eliminating all `cv2.imshow()` dependencies and setting `QT_QPA_PLATFORM=offscreen` guarantees 100% crash-free execution across all

automated evaluation environments.

9. Implementation Details

The project is implemented across 8 focused modules within src/:

1. `detector.py`: Encapsulates `VehicleDetector` supporting dual backends (yolo and mog2). Implements confidence filtering, COCO class mapping, and morphological cleanup.
2. `tracker.py`: Encapsulates `MultiObjectTracker` implementing class-aware bipartite cost matching, Euclidean centroid distance gating, and occlusion persistence (`max_missed=12`).
3. `homography.py`: Encapsulates `HomographyTransformer` using `cv2.getPerspectiveTransform` and `cv2.perspectiveTransform` to project image coordinates into metric ground-plane coordinates.
4. `speed_estimator.py`: Encapsulates `SpeedEstimator` calculating displacement velocities with horizon singularity suppression.
5. `traffic_analyzer.py`: Encapsulates `TrafficAnalyzer` computing macroscopic stream statistics, HCM Level of Service ratings, speed dispersion, and fleet modal splits.
6. `visualizer.py`: Renders resolution-adaptive bounding boxes, trajectory trails, and semi-transparent heads-up display (HUD) banners.
7. `video_processor.py`: Coordinates video decoding, annotation, and streaming to `annotated_video.mp4` and `vehicle_data.csv`.
8. `main.py`: Headless CLI entry point providing argument parsing, fault-tolerant configuration synthesis, and execution summary formatting.

10. Screenshots & Experimental Results

10.1 Empirical Measurement Summary (Chicago Michigan Ave 4K Dataset)

Metric	Calibrated Physical Run	Uncalibrated Baseline Run
Evaluated Video Frames	300	300
Unique Tracked Vehicles	29	29
Average Active Vehicles / Frame	7.62	7.62
Peak Active Vehicles Concurrent	10	10
Traffic Density (veh/lane-km)	43.87	43.87
HCM Level of Service (LOS)	LOS F (Congested Arterial)	Moderate Density
Total Raw Detections	2,285	2,285
Passenger Car Detections	1,915 (83.8%)	1,915 (83.8%)
Public Transit Bus Detections	351 (15.4%)	351 (15.4%)
Commercial Truck Detections	19 (0.8%)	19 (0.8%)
Mean Operating Velocity	9.08 m/s (32.70 km/h)	301.92 px/s
85th-Percentile Speed (V_{85})	15.21 m/s (54.77 km/h)	512.4 px/s
15th-Percentile Speed (V_{15})	1.14 m/s (4.10 km/h)	38.2 px/s
Speed Standard Deviation (σ_v)	7.59 m/s	184.2 px/s
Speed Coefficient of Variation (CV)	1.01	0.61
Speed Calibration Status	Active (config/calibration.json)	Disabled

10.2 Visual Detection & Tracking Overlay

The visual output combines bounding boxes, vehicle identities, physical velocity tags, historical motion paths, and a semi-transparent HUD banner:

11. Testing Approach

Testing is automated using `pytest`, structured in tests/:

- * `test_iou` & `test_iou_cases`: Boundary testing of spatial intersection-over-union algorithms.
- * `test_track_persistence`: Temporal association testing ensuring consistent `track_id` across occlusions.
- * `test_detection_dataclass`: Schema verification for detection records.
- * `test_identity_like_mapping`: Geometric verification of projective homography matrices.
- * `test_pixel_speed_positive`: Kinematic displacement testing in pixel coordinate space.
- * `test_calibrated_speed_estimator`: Mathematical verification of physical metric velocity conversions.
- * `test_traffic_analyzer_congestion_and_calibrated`: Statistical aggregation testing for velocity, class volumes, and congestion flags.

- * test_hcm_level_of_service: Highway Capacity Manual Level of Service criteria validation.
- * test_speed_percentiles: Verification of V_{85} and V_{15} linear interpolation algorithms.
- * test_mog2_detector_initialization: Initialization and morphological structuring verification for the MOG2 background subtractor.
- * test_configuration_fallback_resilience: Validates that non-existent calibration paths gracefully fall back without raising unhandled exceptions.

Test Results: 13 passed in 0.11s (100% test pass rate).

12. Challenges Faced & Solutions

1. Horizon Projective Singularity:

***Challenge*:** Perspective homography maps ground points near the vanishing horizon toward infinity, producing extreme velocity spikes for newly appearing distant vehicles.

***Solution*:** Implemented an upper-bound physical velocity filter ($v \leq 38.0$ m/s ≈ 136.8 km/h) alongside requiring minimum track history length before calculating velocity.

2. Headless Execution Compatibility:

***Challenge*:** Automated evaluation sandboxes lack display servers; invoking GUI functions like `cv2.imshow()` causes immediate crashes.

***Solution*:** Enforced strict CLI execution by stripping GUI window calls, setting `QT_QPA_PLATFORM=offscreen`, and writing all annotated video directly to MP4 containers.

3. Bounding-Box Centroid Jitter:

***Challenge*:** Small frame-to-frame detector bounding-box fluctuations caused artificial velocity noise.

***Solution*:** Designed a multi-frame sliding temporal window ($N=5$) that smooths centroid positions over discrete time intervals.

4. Configuration File Fragility:

***Challenge*:** Evaluator grading scripts may run without specifying calibration files or provide invalid paths.

***Solution*:** Implemented a fault-tolerant loader that automatically locates default configurations or dynamically synthesizes valid road geometry based on video dimensions.

13. Learnings & Key Takeaways

***Geometric Foreshortening Rectification:** Gained deep practical mastery over 2D planar homography, Direct Linear Transformation (DLT), and the necessity of geometric perspective correction for converting video pixels into physical SI units.

***Semantic vs. Motion Representations:** Hands-on comparison demonstrated that while MOG2 is computationally inexpensive, it fails when vehicles stop at signals; YOLO provides semantic persistence regardless of vehicle motion state.

***Transportation Engineering Metrics:** Learned to apply civil engineering standards (HCM Level of Service, 85th-percentile design speed) to computer vision analytics, bridging machine learning with real-world infrastructure planning.

***Production-Grade Software Engineering:** Emphasized fault-tolerant configuration loading, clean headless design, decoupled architecture, and automated test coverage.

14. Future Enhancements

1. Kalman Filter & DeepSORT Integration: Integrating a Kalman filter motion model with deep appearance ReID embeddings to improve long-term re-identification across wide-area camera handoffs.

2. Multi-Camera Network Tracking: Extending the pipeline to track vehicles across overlapping and non-overlapping multi-camera surveillance networks.

3. Automated Horizon & Vanishing Point Calibration: Utilizing Hough line transforms on road markings to automatically detect vanishing points and compute homography matrices without manual landmark annotation.

4. Edge Device Optimization (TensorRT / ONNX Runtime): Exporting the YOLO11 model to INT8 TensorRT engines for deployment on embedded edge hardware (NVIDIA Jetson).

15. References

1. Szeliski, R. (2022). *Computer Vision: Algorithms and Applications** (2nd ed.). Springer.
2. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing** (4th ed.). Pearson.
3. Transportation Research Board. (2016). *Highway Capacity Manual: A Guide for Multimodal Mobility Analysis** (6th ed.). National Academies of Sciences, Engineering, and Medicine.
4. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection**. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
5. Ultralytics. (2024). *YOLO11: State-of-the-Art Real-Time Object Detection and Tracking**. <https://docs.ultralytics.com>

6. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.